

# Python Set

Sets are used to store multiple items in a single variable. Set is one of 4 built-in data types in Python used to store collections of data, the other 3 are List, Tuple, and Dictionary, all with different qualities and usage.

A set is a collection which is *unordered*, *unchangeable\**, and *unindexed*.

\* **Note:** Set *items* are unchangeable, but you can remove items and add new items.

```
thisset = {"apple", "banana", "cherry"}  
print(thisset)
```

**Output:**

```
{'apple', 'banana', 'cherry'}
```

## Duplicates Not Allowed

Sets cannot have two items with the same value.

```
thisset = {"apple", "banana", "cherry", "apple"}  
print(thisset)
```

**Output:**

```
{'banana', 'cherry', 'apple'}
```

## The set() Constructor

It is also possible to use the set() constructor to make a set.

```
thisset = set(("apple", "banana", "cherry"))  
print(thisset)
```

**Output:**

```
{'banana', 'cherry', 'apple'}
```

## Access Items

You cannot access items in a set by referring to an index or a key, but you can loop through the set items using a 'for' loop, or ask if a specified value is present in a set, by using the in keyword.

```
thisset = {"apple", "banana", "cherry"}  
for x in thisset:  
    print(x)
```

**Output:**

```
apple  
banana  
cherry
```

## Set Methods

### add()

The add() method adds an element to the set. If the element already exists, the add() method does not add the element.

#### Syntax

```
set.add(elmnt)
```

```
thisset = {"apple", "banana", "cherry"}  
thisset.add("orange")  
print(thisset)
```

#### Output:

```
{'apple', 'banana', 'cherry', 'orange'}
```

### clear()

The clear() method removes all elements in a set.

#### Syntax

```
set.clear()
```

```
thisset = {"apple", "banana", "cherry"}  
thisset.clear()  
print(thisset)
```

#### Output:

```
set()
```

### discard()

The discard() method removes the specified item from the set. This method is different from the remove() method, because the remove() method *will raise an error* if the specified item does not exist, and the discard() method *will not*.

#### Syntax

```
set.discard(value)
```

```
thisset = {"apple", "banana", "cherry"}  
thisset.discard("banana")  
print(thisset)
```

#### Output:

```
{'apple', 'cherry'}
```

### pop()

The pop() method removes a random item from the set. This method returns the removed item.

#### Syntax

```
set.pop()
```

```
fruits = {"apple", "banana", "cherry"}
fruits.pop()
print(fruits)
```

**Output:**

```
{'apple', 'banana'}
```

**remove()**

The remove() method removes the specified element from the set.

This method is different from the discard() method, because the remove() method *will raise an error* if the specified item does not exist, and the discard() method *will not*.

```
fruits = {"apple", "banana", "cherry"}
fruits.remove("banana")
print(fruits)
```

**Output:**

```
{'apple', 'cherry'}
```

**copy()**

The copy() method copies the set.

**Syntax**

```
set.copy()
```

```
fruits = {"apple", "banana", "cherry"}
x = fruits.copy()
print(x)
```

**Output:**

```
{'banana', 'cherry', 'apple'}
```

**issubset()**

The issubset() method returns True if all items in the set exists in the specified set, otherwise it returns False.

**Syntax**

```
set.issubset(set)
```

```
x = {"a", "b", "c"}
y = {"f", "e", "d", "c", "b", "a"}
z = x.issubset(y)
print(z)
```

**Output:**

```
True
```

### **issuperset()**

The issuperset() method returns True if all items in the specified set exists in the original set, otherwise it returns False.

#### **Syntax**

```
set.issuperset(set)
```

```
x = {"a", "b", "c"}  
y = {"f", "e", "d", "c", "b", "a"}  
z = y.issuperset(x)  
print(z)
```

#### **Output:**

True

### **isdisjoint()**

The isdisjoint() method returns True if none of the items are present in both sets, otherwise it returns False.

#### **Syntax**

```
set.isdisjoint(set)
```

```
x = {"apple", "banana", "cherry"}  
y = {"google", "microsoft", "apple"}  
z = x.isdisjoint(y)  
print(z)
```

#### **Output:**

False

### **union()**

The union() method returns a set that contains all items from the original set, and all items from the specified set(s). You can specify as many sets you want, separated by commas. It does not have to be a set, it can be any iterable object. If an item is present in more than one set, the result will contain only one appearance of this item.

#### **Syntax**

```
set.union(set1, [set2...])
```

```
x = {"a", "b", "c"}  
y = {"f", "d", "a"}  
z = {"c", "d", "e"}  
result = x.union(y, z)  
print(result)
```

#### **Output:**

```
{'c', 'a', 'd', 'f', 'e', 'b'}
```

## **update()**

The update() method updates the current set, by adding items from another set (or any other iterable).

If an item is present in both sets, only one appearance of this item will be present in the updated set.

### **Syntax**

```
set.update(set)
```

```
x = {"apple", "banana", "cherry"}
y = {"google", "microsoft", "apple"}
x.update(y)
print(x)
```

### **Output:**

```
{'banana', 'google', 'cherry', 'microsoft', 'apple'}
```

## **intersection()**

The intersection() method returns a set that contains the similarity between two or more sets.

Meaning: The returned set contains only items that exist in both sets, or in all sets if the comparison is done with more than two sets.

### **Syntax**

```
set.intersection(set1, [set2 ... etc])
```

```
x = {"a", "b", "c"}
y = {"c", "d", "e"}
z = {"f", "g", "c"}
result = x.intersection(y, z)
print(result)
```

### **Output:**

```
{'c'}
```

## **intersection\_update()**

The intersection\_update() method removes the items that are not present in both sets (or in all sets if the comparison is done between more than two sets).

The intersection\_update() method is different from the intersection() method, because the intersection() method *returns a new set*, without the unwanted items, and the intersection\_update() method *removes* the unwanted items from the original set.

### **Syntax**

```
set.intersection_update(set1, [set2 ... etc])
```

```
x = {"apple", "banana", "cherry"}
y = {"google", "microsoft", "apple"}
x.intersection_update(y)
print(x)
```

### **Output:**

```
{'apple'}
```

### **difference()**

The difference() method returns a set that contains the difference between two sets.

Meaning: The returned set contains items that exist only in the first set, and not in both sets.

#### **Syntax**

```
set.difference(set)
```

```
x = {"apple", "banana", "cherry"}  
y = {"google", "microsoft", "apple"}  
z = x.difference(y)  
print(z)
```

#### **Output:**

```
{'banana', 'cherry'}
```

### **difference\_update()**

The difference\_update() method removes the items that exist in both sets.

The difference\_update() method is different from the difference() method, because the difference() method *returns a new set*, without the unwanted items, and the difference\_update() method *removes* the unwanted items from the original set.

#### **Syntax**

```
set.difference_update(set)
```

```
x = {"apple", "banana", "cherry"}  
y = {"google", "microsoft", "apple"}  
x.difference_update(y)  
print(x)
```

#### **Output:**

```
{'banana', 'cherry'}
```

### **symmetric\_difference()**

The symmetric\_difference() method returns a set that contains all items from both set, but not the items that are present in both sets.

Meaning: The returned set contains a mix of items that are not present in both sets.

#### **Syntax**

```
set.symmetric_difference(set)
```

```
x = {"apple", "banana", "cherry"}  
y = {"google", "microsoft", "apple"}  
z = x.symmetric_difference(y)  
print(z)
```

#### **Output:**

```
{'cherry', 'microsoft', 'banana', 'google'}
```

### **symmetric\_difference\_update()**

The `symmetric_difference_update()` method updates the original set by removing items that are present in both sets, and inserting the other items.

#### **Syntax**

```
set.symmetric_difference_update(set)
```

```
x = {"apple", "banana", "cherry"}  
y = {"google", "microsoft", "apple"}  
x.symmetric_difference_update(y)  
print(x)
```

#### **Output**

```
{'banana', 'google', 'microsoft', 'cherry'}
```